KAIROS BY EXOUSIA
DEVELOPER HANDOVER

EVERYTHING YOU NEED TO CHANGE THIS CODEBASE SAFELY

Building on Kairos

Architecture, the invariants that must survive your change, how the suites are run, and the outstanding work written as tasks rather than aspirations. Read sections 01–05 before your first commit; the rest is reference.

19 AUGUST 2026 · COUNTED FROM THE REPOSITORY ON THE DAY IT WAS WRITTEN

00 CONTENTS

01 SHAPE

One process, two halves, no framework surprises

183 API ENDPOINTS	30 ROUTE MODULES
42 TABLES	31 CLIENT ROUTES
51 BROWSER SUITES	~23k LINES OF APP CODE

Server: Express 4, plain CommonJS, no ORM, no build step. Sessions are cookies; passwords are scrypt. **Client:** React 18 + Vite + `react-router-dom` v7, no UI framework — a hand-rolled design system in one stylesheet. In production the server serves the built client and hands unknown paths back to `index.html`.

```
app/
  server/
    index.js        mounting, the 503 gate, /api/status, the SPA fallback
    schema.sql      runs on every boot; CREATE TABLE IF NOT EXISTS only
    lib/            40 modules — the decisions live here, not in routes
    routes/         30 routers — validation, authorisation, and shape
  client/
    src/pages/      screens, incl. dashboard/ pa/ spaces/ onboarding/
    src/components/ AppShell, SoonButton, TimezonePicker, forms
    src/lib/        api client, auth context, timezone helpers
```

Listen first, then be ready

`index.js` binds the port immediately and only then brings the database up. Every `/api` route is held at 503 until the schema is ready; `/api/status` answers throughout and names the problem. The page itself always loads.

> WHY THIS ORDER
>
> The old order made a database problem look like a deploy problem: nothing listened, so the platform showed a successful build followed by a deploy spinning forever with no explanation, and the only way to learn anything was to read a log that stayed silent. Coming up and saying what is wrong is strictly more useful than dying quietly. Never move database work in front of `listen()`.

Async routers

Express 4 does not catch a rejected promise from a handler — the rejection becomes an unhandled rejection and the request hangs until the client gives up. Since every route awaits the database, that would be the normal failure mode. So routers are created through `lib/asyncRouter.js`, which wraps each handler and forwards rejections to `next()`, preserving the four-argument shape for error middleware.

Use `asyncRouter()` for any new router. A bare `express.Router()` will look fine in review and hang in production.

Body limits

100 KB globally, with one carve-out: the voice endpoint skips the global parser and declares its own ceiling. A parser mounted on the route cannot help — the global one runs first and rejects the body before the route's limit is ever consulted, which showed up as a 500 on any note longer than a few seconds.

---

02 THE ADAPTER

One interface, two backends

`lib/db.js` is the only file that knows which database is in use. `DATABASE_URL` set means Postgres via `pg`; unset means a local SQLite file via Node 22's built-in `node:sqlite`, so local development needs no database installed.

```
const row  = await db.prepare('SELECT * FROM users WHERE id = ?').get(id);
const rows = await db.prepare('SELECT * FROM trips WHERE owner_id = ?').all(id);
await db.prepare('INSERT INTO trips (...) VALUES (?, ?, ?)').run(a, b, c);
```

Same shape both ways, always awaited. Route code never learns which backend it is talking to, and the SQL stays in the portable subset both dialects accept. Placeholders are `?` everywhere; the Postgres implementation rewrites them.

Things the adapter absorbs

- `PRAGMA` and `INSERT OR IGNORE` are translated.
- Transactions go through the adapter, not raw `BEGIN` / `COMMIT`.
- Column-existence checks are scoped to `current_schema()` — without that, a foreign `public.users` would make the migrations think our columns already existed and skip them.
- A malformed `DATABASE_URL` is diagnosed by shape before anything tries to connect, and reported as a database failure rather than thrown at require time — throwing there would kill the process before it binds a port.
- Connection is retried on transient errors only (`ECONNREFUSED`, `ETIMEDOUT` and friends). Bad credentials or a syntax error in our own schema fail immediately, because retrying will not fix them.

DATABASE_SCHEMA

Optional, Postgres only. Keeps every table in a named schema instead of `public`, so Kairos can share one free Postgres instance with another app without its `users` table colliding. The value must be a plain lowercase identifier — it reaches the server as a connection startup option and as `CREATE SCHEMA`, neither of which takes a bind parameter, so anything else is refused at boot rather than escaped.

---

03 MIGRATIONS

Additive, idempotent, and in two different places

This is the rule people get wrong most often, so learn it before you touch the schema.

WHAT YOU ARE ADDING	WHERE IT GOES
A new **table**	`schema.sql`, as `CREATE TABLE IF NOT EXISTS`
A new **column on an existing table**	`lib/db.js`, in `ready()`, via `ensureColumn()`
An index on a **new** table	`schema.sql`
An index on a **retrofitted** column	`lib/db.js`, via `ensureIndex()`, after the `ensureColumn`

> WHY A COLUMN CANNOT GO IN SCHEMA.SQL
>
> `schema.sql` runs on every boot. For a table that already exists, `CREATE TABLE IF NOT EXISTS` is a no-op — so a column added inside that statement appears on a fresh database and silently never appears on an existing one. Every deployment that has ever run is an existing database. There are currently 36 retrofitted columns and 5 retrofitted indexes; add yours the same way.

```
// in ready(), after the schema has run
await ensureColumn('itinerary_items', 'pickup_token', 'TEXT');
await ensureIndex('idx_itin_pickup', 'itinerary_items (pickup_token)');
```

There is no migration table and no down-migration. Everything is additive and idempotent, so booting an old database and a new one converge on the same shape. If you ever need a destructive change, that is a conversation before it is a commit.

`bupgrade` proves an existing database is upgraded in place losing nothing, and `bschema` proves schema isolation. Both must stay green.

04 ACCESS

Four separate systems, deliberately not one

QUESTION	ANSWERED IN	FAILURE CODE
May this user act for this principal?	`lib/paAccess.js`	403

QUESTION	ANSWERED IN	FAILURE CODE
May this user see this space or thread?	`lib/spaceAccess.js`	`404`
May this user see this *field*?	`lib/essentials.js` + `lib/stepUp.js`	`403 / step-up`
Is this account entitled to this feature?	`lib/plans.js`	`402-ish, and off`

> **403 VERSUS 404 IS NOT A STYLE CHOICE**
>
> `requirePaAccess` returns **403** — app-wide convention, and correct: the principal's existence is not a secret, the URL contains their id, and they were named to you by somebody. `spaceAccess` returns **nothing**, so the route answers **404** — the existence of a private space *is* the secret. Do not "harmonise" these. Each is right where it is.

The structural preference

Where a door can be made not to exist rather than merely locked, make it not exist.

- A private space short-circuits *before* consulting `space_members`, so a stray row from a future migration cannot open it.
- Household staff live in `household_members`, not `memberships` — because `requirePaAccess` grants on any active membership whatever the role, and five other queries do the same. A household role on that table would have handed a cook the approval queue, contacts, briefs and the vault at six call sites at once, none of which would have looked wrong in review.
- Peer connections live in `connections`, so no query can mistake a peer for a delegate.

When you add a relationship type, ask first whether it belongs in an existing table. The answer is usually no, and the reason is always the same: existing queries.

---

05 INVARIANTS

Things your change must not undo

Each of these is enforced by at least one suite. If your change makes one of them false, the change is wrong — not the test.

> The server binds its port before it touches the database.
> And answers `/api/status` throughout. `bup`, `bheal`, `bfail`.

> A space you cannot see is indistinguishable from one that does not exist.
> And a private space has no readable members at all. `brooms`.

> A record is immutable once filed.
> Notes are editable; promoting a note to a record is a one-way act. `bmulti`, `bdone`.

> An anchor never moves.
> And nothing after it moves either. `blate`.

> Nothing is sent on somebody's behalf automatically.
> The cascade returns people-to-tell; a human sends. AI Assist returns candidates; a human books.

> Sensitive fields are refused without a key, and revealed only after a step-up.
> Where two-factor is enrolled, the step-up asks for the *code*, not the password. `bnokey`, `bstepup`, `bvault`, `bscope`.

> Two-factor is not demanded at sign-in by default.
> It is spent on the vault. Changing this without the principal asking locks people out of their own accounts — we did it once. `bsignin`, `bmfa`.

> Entitlement never gates reading, and fails open.
> `bplan`.

> Every unbuilt capability is visible, named, inert, and removes itself when it works.
> `bsoon`.

> Nobody's name is displayed in an arrivals hall.
> The driver card shows the flight, the meeting point, the phrase and the signal — never the principal. `bsignal`, `btrip`.

06 THE MAP

Where the load-bearing decisions live

Routes validate and authorise; `lib/` decides. If you find yourself putting a rule in a route, check whether the same rule already exists in a lib and is about to be duplicated — that is how the two paths for scheduling (principal and assistant) diverged the first time.

MODULE	OWNS
db.js	The only file that knows the backend. Migrations, retries, schema isolation.
cascade.js	The delay engine. Gap-absorbs and anchor-never-moves.
spaceAccess.js	Every read and write of a space, thread or message. No second path.
paAccess.js	Assistant reach, and the separately revocable scheduling remit.
essentials.js	The field catalogue and sensitivity. Add a field here, once.
secretBox.js	AES-256-GCM. Key from the environment, never the database.
stepUp.js	Which factor to demand, and the short grace window.
totp.js	RFC 6238, written out. No dependency in the auth path.
household.js	Staff, structurally separate from assistants.
trips.js	Journeys, per-leg zones, arrangements.
pickup.js · pickupSignal.js	The phrase, and the HMAC'd colour-and-shape signal.
visas.js	Coverage, computed. Requirement, refused.
travelTime.js	Maps adapter, traffic-aware, cached, never throws, never rewrites.
directLine.js	The one room per principal.
voiceNotes.js	Encrypted, expiring, hard-capped audio.
stageStatus.js	Records override a stored stage status.
reminders.js	The sweep, and once-per-threshold nudges.
accessCodes.js	Armed pairing codes; identical failures.
handles.js	Normalisation, reserved names, relationship-scoped resolution.
connections.js	Peers. Emphatically not delegation.

MODULE	OWNS
`plans.js`	Entitlement. Fails open, records reach, off by default.
`connectors.js`	The catalogue of 17. Deployment versus account.
`capabilities.js`	The single registry of what does not work yet.
`accountDeletion.js`	Explicit walk of non-cascading authorship columns, in one transaction.
`rateLimit.js`	In memory. Per account *and* per address.
`emailProviders.js`	SendGrid or Resend, one shape, different error envelopes.
`asyncRouter.js`	Rejections become 500s instead of hangs.

07 THE CLIENT

One shell, deep-linkable tabs, no framework

`AppShell` provides the nav rail, the principal switcher, the account menu, the back control and the badge fetches. Every signed-in screen uses it. Tabs are query parameters, so `/dashboard?tab=essentials` is a real address you can send somebody.

Styling is one hand-rolled stylesheet with a small token set. There is no component library and no CSS-in-JS; match the surrounding conventions rather than introducing a third way.

Two client-side pieces worth knowing

- `components/SoonButton.jsx` — the in-situ placeholder for an unbuilt capability. It fetches the registry once per page and renders *nothing* when the capability reports available, so the real control drops into exactly that position with no other change anywhere.
- `components/TimezonePicker.jsx` — searchable over city, region, zone and offset, with an alias map for the cities people actually name, and five prominence bands so typing "lon" offers London before Longyearbyen.

> A TIMEZONE BUG THAT WILL BITE YOU AGAIN
>
> Wall-clock times entered on a screen are in the leg's zone, not the browser's. `src/lib/timezones.js` mirrors `server/lib/timezone.js` line for line, including the two-pass DST offset guess. If you change one, change the other, and add a case to `btz`.

---

08 ADAPTERS

The pattern for anything outside the process

Every external service follows the same four-part shape. Copy it rather than inventing a fifth.

1. **`isConfigured()`** reads the environment and answers honestly.
2. **The call itself** never throws into a request path — it returns a result that says what happened, including a failure.
3. **A `*_BASE_URL` override** so a suite can point it at a local stub. This is why `btravel` and `bsend` can test the real code path without a real vendor.
4. **An entry in the capability registry** if a user-facing control depends on it, listing the exact variables it awaits.

Existing examples: email (SendGrid or Resend), maps (Google Distance Matrix), and the unconfigured seams for calendar sync, WhatsApp, flight data, storage and transcription. The email pair is the best one to read first — the two providers differ mostly in the details that break silently (SendGrid answers 202 with an empty body and reports `{errors:[{message}]}` where Resend uses `{message}`), which is exactly the class of bug the delivery-status work exists to prevent.

---

09 CAPABILITIES

How to add — or retire — an unbuilt feature

`lib/capabilities.js` is the single registry. Each entry carries an id, the control's name, the screen it appears on, a plain-English description, the variables it awaits, a state, and — the important part — **`available`, computed by calling the same function the feature itself calls**.

```
add({
  id: 'travel_time',
```

```
  control: 'Travel time',
  screen: 'itinerary',
  label: 'Travel time with live traffic',
  what: 'Ask the road how long this leg takes, at the hour it happens.',
  available: travelTime.isConfigured(),   // not a hardcoded false
  needs: ['MAPS_API_KEY'],
  state: 'needs_key',
});
```

Then place `<SoonButton feature="travel_time" />` where the working control will live. When the feature starts working, `available` flips, the placeholder renders nothing, and the row disappears from `/coming` — all from one fact on the server. **You delete nothing.**

> NEVER HARDCODE AVAILABILITY
>
> A screen that hardcodes "coming soon" keeps saying it after the credential is set. A screen that hardcodes nothing quietly offers a button that does nothing. Both are how a product loses the right to be believed about anything else.

`state` is `'soon'` when the work is ours and `'needs_key'` when the deployment is missing something an operator can set today. The distinction is for the reader: one is being asked to wait, the other is being handed a task.

---

10 PLANS

Entitlement, switched off, and recording

```
const DEFAULT_PLAN = PLANS[process.env.DEFAULT_PLAN] ? process.env.DEFAULT_PLAN : 'founding';
const ENFORCED = String(process.env.PLAN_ENFORCEMENT || '').toLowerCase() === 'on';
```

`requirePlan(feature)` records that the feature was reached, and refuses only when `ENFORCED`. It fails open on anything unexpected — a null column, a half-run migration, an unknown plan name — and it is only ever applied to *creating* things, never to reading them.

`founding` ranks equal to Executive and is the default, so nothing is withheld from anybody today. Early accounts keep it permanently; that is a promise, not an oversight.

> TWO THINGS TO CHECK BEFORE SHIPPING A PLAN CHANGE
>
> That signup writes `DEFAULT_PLAN` explicitly — a column default alone made the variable inert once, because signup never wrote the field and every account fell to `'founding'` regardless. And that no entitlement check has crept onto a read path; `bplan` asserts that nothing is locked today.

---

11 TESTS

Fifty-one suites, and what each of them is for

Tests are standalone Node scripts, not a framework. Each is named `b*.js`, prints ticks and crosses, exits non-zero on failure, and ends with a single sentence saying what it proved. API suites use `fetch` against a server the suite starts on its own port; UI suites drive Chromium through `playwright-core` at `/opt/pw-browsers/chromium`.

```
bash allsuites.sh          # every suite, sequentially, one port each
node btravel.js            # one suite
bash pgmatrix.sh           # pairing codes across all four storage configs
bash sigmatrix.sh          # signal + trips across the Postgres configs
```

The four database configurations

CONFIG	PROVES
SQLite (no `DATABASE_URL`)	Local development works with no database installed
Postgres, fresh database	The schema builds correctly from nothing
Postgres, `DATABASE_SCHEMA=kairos`	Isolation — Kairos can share an instance with another app
Postgres, an older database	The migrations actually run, losing nothing

Anything touching the schema or a query must be run against all four. The fourth is the one that catches a column added in the wrong place, and it is the one people skip.

Writing a suite

- Assert on *behaviour a person would notice*, and make the sentence at the end say what was proved. "Nobody held up a name, and nobody had to guess" is a better assertion set than "signal endpoint returns 200".
- Scope every fixture to the run — a fixed handle or email means the suite passes exactly once per database.
- Wait on the thing itself, not on a sleep. A pause long enough today is a flake tomorrow.
- Give the suite its own port.
- When a suite fails, work out which of the two is wrong before changing either. Several entries in section 14 exist because the test was right.

---

12 STATUS
19 AUG 2026

The real result of the last full run

All 51 suites, run back to back on this machine: **50 passed, 1 failed.** The one failure is named here rather than filed under "known environmental failure", which is a phrase that hides bugs.

SUITE	WHAT HAPPENS	STATE
`bfail`	Six assertions about database failure modes — a wrong password, a missing database, a garbled URL, a working one. Telling those apart needs a real Postgres at `127.0.0.1:5432`, and this container has none, so every assertion in it is unanswered rather than answered wrongly.	NEEDS POSTGRES

Stand a local Postgres up and this is the fifty-first. Until then, describe the suite as **50 of 51 with the one accounted for** — not as green.

Why every suite now owns its server

Three suites used to share a long-running server on port 4000, and all three failed intermittently. The rate limiter got the blame for a while — it is in memory and keyed on the address as well as the account, so one suite could exhaust another's budget — but that was a symptom sitting on top of something worse.

Twenty-seven of the fifty-one suites delete `app/server/data/kairos.sqlite` before they start. A server that outlives one of those deletions goes on writing to a file that is no longer on disk. The next suite to use it sees a signup succeed and then finds no account — a failure that points nowhere near its cause, and which lands on whichever suite happens to run next in alphabetical order rather than on anything to do with what it tests.

So there is no shared server any more. Each suite spawns one on its own port with its own environment and stops it again, which means run order can no longer change the result. If you add a suite, do the same — and if you reach for a wipe, ask first whether run-scoped fixtures would do instead. They almost always would.

> WAIT FOR THE DATABASE, NOT FOR THE PORT
>
> A suite's readiness loop must poll `/api/status` until `databaseReady` is true. The server binds its port *before* the database is up — deliberately, so a database problem is visible at a URL instead of invisible in a deploy queue — and every API route answers 503 until it is ready. A loop that stops at the first HTTP response hands the suite a server that refuses its signup.

What `bcustody` was, and what fixing it taught

It used to fail every time. Four separate things were wrong, and only the last was about the product at all — which is the useful part of the story, because three of them are mistakes any suite here can make again.

- **It borrowed a server.** That meant borrowing an in-memory rate limiter keyed on the address as well as the account — and a successful login clears the account key, never the address key — so every login any other suite made from `127.0.0.1` counted against it. It also meant borrowing that server's `ENCRYPTION_KEY`, usually absent, so the whole vault half was quietly exercising the refusal path.
- **Its handle was the literal `ada-boss`.** Handles are unique app-wide, so the second run against any database got "that handle is already taken" — which reads exactly like a normalisation bug and is not one.
- **Its expiry assertions were pinned to dates in 2026.** They were going to start failing on a Tuesday for no visible reason.
- **One assertion was wrong about the product.** It demanded that assembling a travel block always costs a password. It does not, and should not: the earlier reveal had already stepped the session up, and the grace window is deliberate. The suite now sets the window to two seconds and checks both halves — the next sensitive read inside it costs nothing, and once it lapses the gate comes back.

THE RULE THAT FALLS OUT OF ALL FOUR

A suite should own its server, scope every fixture to the run, and express dates relative to now. Any suite that does all three can be run twice in a row, in any order, against any database — and a red result from it means something.

13 THE WORK

Outstanding, as tasks

Ordered by what unblocks the most. Each says what exists already, so nobody starts from zero.

TASK	WHAT ALREADY EXISTS	WHAT IS MISSING	
A local Postgres for `bfail`	The suite, and the six failure modes it asserts on. Everything else in the matrix already runs.	A Postgres at `127.0.0.1:5432`. Without one, `bfail` cannot tell a wrong password from a missing database, which is the entire thing it exists to check — and the four-configuration matrix cannot be run at all.	FIRST
Stop the remaining database wipes	Twenty-seven suites that delete `kairos.sqlite` on the way in, and `bcustody` as the worked example of not needing to.	Run-scoped fixtures in each of them, so a suite stops destroying development data as a side effect of running. Now that nothing shares a server this is hygiene rather than correctness, but it is why the bug was so hard to see.	CLEANUP
Live flight status	The connector entry, the capability row, and a cascade that already reasons from anchors.	A flight-data contract, an adapter on the standard shape, and a decision about how a changed departure re-runs the cascade.	CONTRACT
Forward a confirmation	The trip and journey model, the capability row, the placeholder on Trips.	An inbound mail domain, a signed-webhook receiver, and a parser for airline and hotel mail. Start with two airlines and one hotel chain, and refuse anything it cannot read rather than guessing.	BUILD

TASK	WHAT ALREADY EXISTS	WHAT IS MISSING	
Attach a scan	`secretBox`, the essentials catalogue, the placeholder in the vault.	Object storage, an upload path that encrypts before it stores, and a reveal path behind the same step-up as the field it belongs to.	BUILD
Transcribe a voice note	Encrypted, expiring, capped audio, and the placeholder under each note.	A route that does not hand the audio to a third party. When it lands, the text becomes the durable artifact — revisit the thirty-day expiry then, not before.	BUILD
Calendar sync	The seam in `calendarSync.js`, connector entries, honest not-configured everywhere.	OAuth for Google and Microsoft, two-way sync, and a conflict policy. The hard part is not the API — it is deciding what wins when both sides moved.	CREDENTIAL
WhatsApp	The seam in `whatsapp.js` and the connector entry.	A Business API token, template approval, and a rule for what is sent — nothing outbound automatically.	CREDENTIAL
Billing	The whole plan layer, plus recorded reach data telling you what people actually use.	A payment integration, and the switch. Keep the fail-open rule when you throw it.	BUILD
Shared rate-limit store	Per-account and per-address keying, already correct.	A backing store, before running more than one instance. Until then, one instance.	SCALE
Durable Postgres with backups	The adapter, the schema isolation, the migration discipline.	A paid instance. Free tiers have no backups at all — this must happen before a real principal's data lands.	BEFORE LAUNCH
External security review	An access model designed to be reviewable, and suites that state their invariants.	Somebody outside this team, before custody is sold to anyone.	BEFORE LAUNCH
Visa requirement lookup	Coverage, the adapter seam, and a screen that says plainly it is not answered.	A maintained, licensed ruleset. Do not build this from scraped data. If we cannot buy accuracy, the honest answer stays "we do not know".	REFUSED

14 TRAPS

Mistakes already made here, so they are not made twice

> An edit that cannot fail loudly should not be trusted to have happened.
>
> A scripted string replacement assumed an import that was not there, changed nothing, and reported success. The server then would not boot. If you script an edit, assert on the result.

> A column default is not a migration.
>
> `DEFAULT_PLAN` was inert for a while because signup never wrote the column and the default swallowed every new account. Write the value explicitly.

> A control that reacts must not claim to be disabled.
>
> `SoonButton` was built with `aria-disabled` and an `onClick`. Playwright refused to click it, and it was right: a screen reader would announce "dimmed" and the thing would respond anyway. The button is a disclosure; what is unavailable is the feature behind it.

> Navigating to the page you are already on is a reload.
>
> A "back" test assumed a cold arrival and got a preserved history instead. Open a fresh tab.

> Ranking by substring is not ranking.
>
> Typing "lon" offered Longyearbyen before London. Fixed with prominence bands, not by weakening the test.

> Response shapes are not guessable.
>
> Posting a message returns `{ id }`, not `{ message }`. The direct line is surfaced on `/api/today`, not on its own route. Read the route before asserting on it.

> `pkill -f` matches its own shell.
>
> It has killed this session's commands more than once. Find the PID and kill that.

> A leftover stub server holds its port.
>
> Suites that start a stub must stop it, or the next run fails with `EADDRINUSE` and looks like a hang.

Your first day

```
# two processes
cd app/server && npm install && npm run dev     # :4000
cd app/client && npm install && npm run dev     # :5173, proxies /api

# single-process, as production runs it
cd app/client && npm run build
cd app/server && npm start                        # :4000 serves both

# a key, if you want the vault and voice notes locally
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Then, in order:

1. Sign up, walk onboarding, book yourself from an incognito window.
2. Build a day with a flight at the end and overrun the first thing by two hours. Read `lib/cascade.js` with the result in front of you.
3. Read `lib/db.js` top to bottom. It is the file most likely to bite you.
4. Read `lib/spaceAccess.js` and `lib/paAccess.js` together, and note that they answer differently on purpose.
5. Run `bash allsuites.sh` once before you change anything, so you know what this machine's baseline looks like.
6. Then read section 05 again. Those are the things a review will ask you about.

> Comments in this codebase explain *why*, not what. When you change a decision, change the comment that argued for it — a stale rationale is worse than none.
